

Positive Sharing and Abstract Machines

Beniamino Accattoli¹, Claudio Sacerdoti Coen², Jui-Hsuan Wu³

Inria & LIX, École Polytechnique¹

Alma Mater Studiorum - Università di Bologna²

CNRS, LIP, ENS de Lyon³

APLAS 2025, IIIT Bangalore, India

28 October 2025

Positive λ -calculus λ_{pos}

A natural abstract machine for λ_{pos}

Slicing to the rescue

Conclusion

Design of λ -calculi with sharing

The λ -calculus has a particularly simple syntax:

$$t, u ::= x \mid tu \mid \lambda x. t$$

Design of λ -calculi with sharing

The λ -calculus has a particularly simple syntax:

$$t, u ::= x \mid tu \mid \lambda x. t \mid \text{let } x = t \text{ in } u$$

To introduce sharing in the λ -calculus is, one can simply add `let` expressions.

Design of λ -calculi with sharing

The λ -calculus has a particularly simple syntax:

$$t, u ::= x \mid tu \mid \lambda x. t \mid u[x \leftarrow t]$$

To introduce sharing in the λ -calculus is, one can simply add `let` expressions (or explicit substitutions, ESs).

Design of λ -calculi with sharing

The λ -calculus has a particularly simple syntax:

$$t, u ::= x \mid tu \mid \lambda x. t \mid u[x \leftarrow t]$$

To introduce sharing in the λ -calculus is, one can simply add `let` expressions (or explicit substitutions, ESs).

How can we design such a calculus?

1. put restrictions on the syntax thanks to ESs.

↪ encode $(tu)r$ as $(xr)[x \leftarrow tu]$ for example

2. choose the style of substitution steps

↪ small-step vs.

$$(xx)[x \leftarrow t] \rightarrow tt$$

micro-step

$$(xx)[x \leftarrow t] \rightarrow (tx)[x \leftarrow t] \rightarrow (tt)[x \leftarrow t]$$

Design of λ -calculi with sharing

The λ -calculus has a particularly simple syntax:

$$t, u ::= x \mid tu \mid \lambda x. t \mid u[x \leftarrow t]$$

To introduce sharing in the λ -calculus is, one can simply add `let` expressions (or explicit substitutions, ESs).

How can we design such a calculus?

1. put restrictions on the syntax thanks to ESs.
 $\hookrightarrow$ encode $(tu)r$ as $(xr)[x \leftarrow tu]$ for example

2. choose the style of substitution steps

$\hookrightarrow$ small-step vs.

$$(xx)[x \leftarrow t] \rightarrow tt$$

micro-step

$$(xx)[x \leftarrow t] \rightarrow (tx)[x \leftarrow t] \rightarrow (tt)[x \leftarrow t]$$

Design of λ -calculi with sharing

The λ -calculus has a particularly simple syntax:

$$t, u ::= x \mid tu \mid \lambda x. t \mid u[x \leftarrow t]$$

To introduce sharing in the λ -calculus is, one can simply add `let` expressions (or explicit substitutions, ESs).

How can we design such a calculus?

1. put restrictions on the syntax thanks to ESs.

↪ encode $(tu)r$ as $(xr)[x \leftarrow tu]$ for example

2. choose the style of substitution steps

↪ small-step

vs.

micro-step

$$(xx)[x \leftarrow t] \rightarrow tt$$

$$(xx)[x \leftarrow t] \rightarrow (tx)[x \leftarrow t] \rightarrow (tt)[x \leftarrow t]$$

Design of λ -calculi with sharing

The λ -calculus has a particularly simple syntax:

$$t, u ::= x \mid tu \mid \lambda x. t \mid u[x \leftarrow t]$$

To introduce sharing in the λ -calculus is, one can simply add `let` expressions (or explicit substitutions, ESs).

How can we design such a calculus?

1. put restrictions on the syntax thanks to ESs.

↪ encode $(tu)r$ as $(xr)[x \leftarrow tu]$ for example

2. choose the style of substitution steps

↪ small-step

vs.

micro-step

$$(xx)[x \leftarrow t] \rightarrow tt$$

$$(xx)[x \leftarrow t] \rightarrow (tx)[x \leftarrow t] \rightarrow (tt)[x \leftarrow t]$$

The positive λ -calculus (λ_{pos}) is a "micro-step" call-by-value λ -calculus with a restricted use of ESs.

Positive λ -calculus λ_{pos}

Syntax: from a proofs-as-terms interpretation of *focused* proofs using the positive polarity assignment to atoms ([Miller and Wu, 2023] and [Wu, 2023]).

Positive λ -calculus λ_{pos}

Syntax: from a proofs-as-terms interpretation of *focused* proofs using the positive polarity assignment to atoms ([Miller and Wu, 2023] and [Wu, 2023]).

Semantics: λ_{pos} captures the essence of *useful sharing* ([Accattoli and Wu, 2024]).

Positive λ -calculus λ_{pos}

Syntax: from a proofs-as-terms interpretation of *focused* proofs using the positive polarity assignment to atoms ([Miller and Wu, 2023] and [Wu, 2023]).

Semantics: λ_{pos} captures the essence of *useful sharing* ([Accattoli and Wu, 2024]).

Bites $b, b' ::= yz \mid \lambda y. u \mid (\lambda y. u)z$

Terms $t, u ::= x \mid t[x \leftarrow b]$

Contexts $O ::= \langle \cdot \rangle \mid O[x \leftarrow b]$

Positive λ -calculus λ_{pos}

Syntax: from a proofs-as-terms interpretation of *focused* proofs using the positive polarity assignment to atoms ([Miller and Wu, 2023] and [Wu, 2023]).

Semantics: λ_{pos} captures the essence of *useful sharing* ([Accattoli and Wu, 2024]).

Bites $b, b' ::= yz \mid \lambda y. u \mid (\lambda y. u)z$

Terms $t, u ::= x \mid t[x \leftarrow b] \quad \text{let } x_1 = b_1 \text{ in } \cdots \text{let } x_n = b_n \text{ in } x_{n+1}$

Contexts $O ::= \langle \cdot \rangle \mid O[x \leftarrow b]$

Positive λ -calculus λ_{pos}

Syntax: from a proofs-as-terms interpretation of *focused* proofs using the positive polarity assignment to atoms ([Miller and Wu, 2023] and [Wu, 2023]).

Semantics: λ_{pos} captures the essence of *useful sharing* ([Accattoli and Wu, 2024]).

Bites $b, b' ::= yz \mid \lambda y. u \mid (\lambda y. u)z$

Terms $t, u ::= x \mid t[x \leftarrow b] \quad \text{let } x_1 = b_1 \text{ in } \dots \text{let } x_n = b_n \text{ in } x_{n+1}$

Contexts $O ::= \langle \cdot \rangle \mid O[x \leftarrow b] \quad \text{let } x_1 = b_1 \text{ in } \dots \text{let } x_n = b_n \text{ in } \langle \cdot \rangle$

Positive λ -calculus λ_{pos}

Syntax: from a proofs-as-terms interpretation of *focused* proofs using the positive polarity assignment to atoms ([Miller and Wu, 2023] and [Wu, 2023]).

Semantics: λ_{pos} captures the essence of *useful sharing* ([Accattoli and Wu, 2024]).

Bites $b, b' ::= yz \mid \lambda y. u \mid (\lambda y. u)z$

Terms $t, u ::= x \mid t[x \leftarrow b] \quad \text{let } x_1 = b_1 \text{ in } \dots \text{let } x_n = b_n \text{ in } x_{n+1}$

Contexts $O ::= \langle \cdot \rangle \mid O[x \leftarrow b] \quad \text{let } x_1 = b_1 \text{ in } \dots \text{let } x_n = b_n \text{ in } \langle \cdot \rangle$

Reduction rules:

$$\begin{array}{lcl} t[x \leftarrow (\lambda y. \overbrace{O\langle z \rangle}^u)w] & \mapsto_m & O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \rangle \\ O\langle t[x \leftarrow yz]\rangle[y \leftarrow \lambda w. u] & \mapsto_e & O\langle t[x \leftarrow (\lambda w. u)z]\rangle[y \leftarrow \lambda w. u] \end{array}$$

Example

$$\begin{array}{lcl} t[x \leftarrow (\lambda y. \overbrace{O\langle z \rangle}^u) w] & \mapsto_{\text{m}} & O\langle t\{x \leftarrow z\} \rangle \{y \leftarrow w\} \\ O\langle t[x \leftarrow yz] \rangle [y \leftarrow \lambda w. u] & \mapsto_{\text{e}} & O\langle t[x \leftarrow (\lambda w. u)z] \rangle [y \leftarrow \lambda w. u] \end{array}$$

Example

$$\begin{array}{lcl} t[x \leftarrow (\lambda y. \overbrace{O\langle z \rangle}^u)w] & \mapsto_m & O\langle t\{x \leftarrow z\} \rangle\{y \leftarrow w\} \\ O\langle t[x \leftarrow yz] \rangle[y \leftarrow \lambda w.u] & \mapsto_e & O\langle t[x \leftarrow (\lambda w.u)z] \rangle[y \leftarrow \lambda w.u] \end{array}$$

Example of reduction:

$$x[x \leftarrow yy][y \leftarrow zz'][z \leftarrow \lambda w.w'[w' \leftarrow ww]]$$

Example

$$\begin{array}{lcl} t[x \leftarrow (\lambda y. \overbrace{O\langle z \rangle}^u)w] & \mapsto_m & O\langle t\{x \leftarrow z\} \rangle\{y \leftarrow w\} \\ O\langle t[x \leftarrow yz] \rangle[y \leftarrow \lambda w.u] & \mapsto_e & O\langle t[x \leftarrow (\lambda w.u)z] \rangle[y \leftarrow \lambda w.u] \end{array}$$

Example of reduction:

$$x[x \leftarrow yy][y \leftarrow \underline{zz'}][\underline{z \leftarrow \lambda w.w'[w' \leftarrow ww]}]$$

Example

$$\begin{array}{lcl} t[x \leftarrow (\lambda y. \overbrace{O\langle z \rangle}^u)w] & \mapsto_m & O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \\ O\langle t[x \leftarrow yz]\rangle[y \leftarrow \lambda w.u] & \mapsto_e & O\langle t[x \leftarrow (\lambda w.u)z]\rangle[y \leftarrow \lambda w.u] \end{array}$$

Example of reduction:

$$\begin{array}{lcl} x[x \leftarrow yy][y \leftarrow zz'][z \leftarrow \lambda w.w'[w' \leftarrow ww]] & & \\ \rightarrow_e & x[x \leftarrow yy][y \leftarrow (\lambda w.w'[w' \leftarrow ww])z'][z \leftarrow \lambda w.w'[w' \leftarrow ww]] & \end{array}$$

Example

$$\begin{array}{lcl} t[x \leftarrow (\lambda y. \overbrace{O\langle z \rangle}^u)w] & \mapsto_m & O\langle t\{x \leftarrow z\} \rangle\{y \leftarrow w\} \\ O\langle t[x \leftarrow yz] \rangle[y \leftarrow \lambda w.u] & \mapsto_e & O\langle t[x \leftarrow (\lambda w.u)z] \rangle[y \leftarrow \lambda w.u] \end{array}$$

Example of reduction:

$$\begin{array}{lcl} x[x \leftarrow yy][y \leftarrow zz'][z \leftarrow \lambda w.w'[w' \leftarrow ww]] & & \\ \rightarrow_e & x[x \leftarrow yy][y \leftarrow \underline{(\lambda w.w'[w' \leftarrow ww])z'}][z \leftarrow \lambda w.w'[w' \leftarrow ww]] & \end{array}$$

Example

$$\begin{array}{lcl}
 t[x \leftarrow (\lambda y. \overbrace{O\langle z \rangle}^u)w] & \mapsto_m & O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \\
 O\langle t[x \leftarrow yz]\rangle[y \leftarrow \lambda w.u] & \mapsto_e & O\langle t[x \leftarrow (\lambda w.u)z]\rangle[y \leftarrow \lambda w.u]
 \end{array}$$

Example of reduction:

$$\begin{array}{lcl}
 & & x[x \leftarrow yy][y \leftarrow zz'][z \leftarrow \lambda w.w'[w' \leftarrow ww]] \\
 \rightarrow_e & & x[x \leftarrow yy][y \leftarrow (\lambda w.w'[w' \leftarrow ww])z'][z \leftarrow \lambda w.w'[w' \leftarrow ww]] \\
 \rightarrow_m & & x[x \leftarrow w'_1 w'_1][w'_1 \leftarrow z' z'][z \leftarrow \lambda w.w'[w' \leftarrow ww]]
 \end{array}$$

Right strategy

Just reduce the redexes from right to left!

Right strategy

Just reduce the redexes from right to left!

This can be formally defined as a relation $\rightarrow_r$ using *right contexts* R .

Example:

- $\langle \cdot \rangle [x \leftarrow yy][z \leftarrow \lambda w.t]$ is a right context
- $\langle \cdot \rangle [x \leftarrow yy][y \leftarrow \lambda w.t]$ is not a right context

Right strategy

Just reduce the redexes from right to left!

This can be formally defined as a relation $\rightarrow_r$ using *right contexts* R .

Example:

- $\langle \cdot \rangle [x \leftarrow yy][z \leftarrow \lambda w.t]$ is a right context
- $\langle \cdot \rangle [x \leftarrow yy][y \leftarrow \lambda w.t]$ is not a right context

Lemma (Basic properties of the right strategy)

1. *Determinism*: if $t \rightarrow_r t_1$ and $t \rightarrow_r t_2$ then $t_1 = t_2$;
2. *No premature stops*: if $t \rightarrow_{\text{pos}} u$ then $t \rightarrow_r r$ for some r .

Natural PPositive Machine (Natural POM)

States are pairs $t \triangleleft R$.

Transitions:

Natural POsitive Machine (Natural POM)

States are pairs $t \triangleleft R$.

Transitions:

$$\begin{array}{llll}
 t[x \leftarrow \lambda y. u] & \triangleleft R & \begin{array}{c} \parallel \\ \rightsquigarrow_{\text{sea}_1} \\ \parallel \end{array} & \begin{array}{c} t \\ t \\ t[x \leftarrow (\lambda w. u)^\alpha z] \\ O\langle t\{x \leftarrow z\}\rangle\{y \leftarrow w\} \end{array} \\
 t[x \leftarrow yz] & \triangleleft R & \begin{array}{c} \parallel \\ \rightsquigarrow_{\text{sea}_2} \\ \parallel \end{array} & \\
 t[x \leftarrow yz] & \triangleleft R & \begin{array}{c} \parallel \\ \rightsquigarrow_e \\ \parallel \end{array} & \\
 t[x \leftarrow (\lambda y. O\langle z \rangle)w] & \triangleleft R & \begin{array}{c} \parallel \\ \rightsquigarrow_m \\ \parallel \end{array} &
 \end{array}
 \quad
 \begin{array}{l}
 \triangleleft R\langle\langle\cdot\rangle[x \leftarrow \lambda y. u]\rangle \\
 \triangleleft R\langle\langle\cdot\rangle[x \leftarrow yz]\rangle \\
 \triangleleft R \\
 \triangleleft R
 \end{array}$$

Complexity of abstract machines

We are interested in the overhead of abstract machines with respect to its underlying calculus/strategy, that is, to the following two parameters:

- the size $|t|$ of the initial term t
- the number of steps of the calculus

Complexity of abstract machines

We are interested in the overhead of abstract machines with respect to its underlying calculus/strategy, that is, to the following two parameters:

- the size $|t|$ of the initial term t
- the number of steps of the calculus

Complexity of abstract machines

We are interested in the overhead of abstract machines with respect to its underlying calculus/strategy, that is, to the following two parameters:

- the size $|t|$ of the initial term t
- the number of steps of the calculus

Complexity of abstract machines

We are interested in the overhead of abstract machines with respect to its underlying calculus/strategy, that is, to the following two parameters:

- the size $|t|$ of the initial term t
- the number of steps of the calculus

Many abstract machines in the literature have been shown to be bi-linear, that is, linear in both parameters.

Complexity of abstract machines

We are interested in the overhead of abstract machines with respect to its underlying calculus/strategy, that is, to the following two parameters:

- the size $|t|$ of the initial term t
- the number of steps of the calculus

Many abstract machines in the literature have been shown to be bi-linear, that is, linear in both parameters.

Unfortunately, Natural POM is not one of them.

Inefficiency of Natural POM

The inefficiency of Natural POM comes from the following transition:

$$t[x \leftarrow (\lambda y. O\langle z \rangle)w] \triangleleft R \rightsquigarrow_m O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \rangle \triangleleft R$$

Inefficiency of Natural POM

The inefficiency of Natural POM comes from the following transition:

$$t[x \leftarrow (\lambda y. O\langle z \rangle)w] \triangleleft R \rightsquigarrow_m O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \rangle \triangleleft R$$

The meta-level renaming $O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \rangle$ is not an issue since y can only appear free in $O\langle z \rangle$, which is always a subterm (up to α -renaming) of the initial term.

Inefficiency of Natural POM

The inefficiency of Natural POM comes from the following transition:

$$t[x \leftarrow (\lambda y. O\langle z \rangle)w] \triangleleft R \rightsquigarrow_m O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \rangle \triangleleft R$$

The meta-level renaming $O\langle t\{x \leftarrow z\}\{y \leftarrow w\} \rangle$ is not an issue since y can only appear free in $O\langle z \rangle$, which is always a subterm (up to α -renaming) of the initial term.

However, the renaming $t\{x \leftarrow z\}$ is problematic: t is not always a subterm of the initial term!

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

$$\frac{\text{ACTIVE CODE}}{x[x \leftarrow yz][z \leftarrow yy][y \leftarrow \lambda y'.t'] \quad \triangleleft \quad \langle \cdot \rangle} \text{RIGHT CTX} \quad \rightsquigarrow_{\text{sea}_1}$$

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

ACTIVE CODE		RIGHT CTX	
$x[x \leftarrow yz][z \leftarrow yy][y \leftarrow \lambda y'.t']$	$\triangleleft$	$\langle \cdot \rangle$	$\rightsquigarrow_{\text{sea}_1}$
$x[x \leftarrow yz][z \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\rightsquigarrow_e$

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

ACTIVE CODE		RIGHT CTX	
$x[x \leftarrow yz][z \leftarrow yy][y \leftarrow \lambda y'.t']$	$\triangleleft$	$\langle \cdot \rangle$	$\leadsto_{\text{sea}_1}$
$x[x \leftarrow yz][z \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_e$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.t'')y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$=$

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

ACTIVE CODE		RIGHT CTX	
$x[x \leftarrow yz][z \leftarrow yy][y \leftarrow \lambda y'.t']$	$\triangleleft$	$\langle \cdot \rangle$	$\leadsto_{\text{sea}_1}$
$x[x \leftarrow yz][z \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_e$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.t'')y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$=$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.x''[x'' \leftarrow y''z''] [z'' \leftarrow y''y''])y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_m$

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

ACTIVE CODE		RIGHT CTX	
$x[x \leftarrow yz][z \leftarrow yy][y \leftarrow \lambda y'.t']$	$\triangleleft$	$\langle \cdot \rangle$	$\leadsto_{\text{sea}_1}$
$x[x \leftarrow yz][z \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_e$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.t'')y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$=$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.x''[x'' \leftarrow y''z''] [z'' \leftarrow y''y''])y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_m$
$x[x \leftarrow yx''] [x'' \leftarrow yz''] [z'' \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_e$

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

ACTIVE CODE		RIGHT CTX	
$x[x \leftarrow yz][z \leftarrow yy][y \leftarrow \lambda y'.t']$	$\triangleleft$	$\langle \cdot \rangle$	$\leadsto_{\text{sea}_1}$
$x[x \leftarrow yz][z \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_e$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.t'')y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$=$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.x''[x'' \leftarrow y''z''] [z'' \leftarrow y''y''])y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_m$
$x[x \leftarrow yx''] [x'' \leftarrow yz''] [z'' \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_e$
$x[x \leftarrow yx''] [x'' \leftarrow yz''] [z'' \leftarrow (\lambda y'''.t''')y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\leadsto_m$

How big is the inefficiency?

Let $t := x[x \leftarrow yz][z \leftarrow yy]$ and $t' := x'[x' \leftarrow y'z'][z' \leftarrow y'y']$ and so on.

ACTIVE CODE		RIGHT CTX	
$x[x \leftarrow yz][z \leftarrow yy][y \leftarrow \lambda y'.t']$	$\triangleleft$	$\langle \cdot \rangle$	$\rightsquigarrow_{\text{sea}_1}$
$x[x \leftarrow yz][z \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\rightsquigarrow_e$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.t'')y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$=$
$x[x \leftarrow yz][z \leftarrow (\lambda y''.x''[x'' \leftarrow y''z''] [z'' \leftarrow y''y''])y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\rightsquigarrow_m$
$x[x \leftarrow yx''] [x'' \leftarrow yz''] [z'' \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\rightsquigarrow_e$
$x[x \leftarrow yx''] [x'' \leftarrow yz''] [z'' \leftarrow (\lambda y''' . t''')y]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\rightsquigarrow_m$
$x[x \leftarrow yx''] [x'' \leftarrow yx'''] [x''' \leftarrow yz'''] [z''' \leftarrow yy]$	$\triangleleft$	$[y \leftarrow \lambda y'.t']$	$\rightsquigarrow_e$
$\vdots$			

Slicing to the rescue

$$t[x \leftarrow (\lambda y. O'\langle z \rangle)]w \triangleleft R \rightsquigarrow_m O'\langle t\{x \leftarrow z\} \rangle\{y \leftarrow w\} \triangleleft R$$

The idea is to delay the merging of t and $O'\langle z \rangle$ as $O'\langle t\{x \leftarrow z\} \rangle$

Slicing to the rescue

$$t[x \leftarrow (\lambda y. O'\langle z \rangle)w] \triangleleft R \rightsquigarrow_m O'\langle t\{x \leftarrow z\} \rangle\{y \leftarrow w\} \triangleleft R$$

The idea is to delay the merging of t and $O'\langle z \rangle$ as $O'\langle t\{x \leftarrow z\} \rangle$
 $\hookrightarrow$ put the pair (**slice**) (t, x) , denoted as $t[x \leftarrow \cdot]$, in a new stack storing delayed merges, and keeping $O'\langle z \rangle\{y \leftarrow w\}$ as the active code.

Slicing to the rescue

$$t[x \leftarrow (\lambda y. O'\langle z \rangle)]w \triangleleft R \rightsquigarrow_m O'\langle t\{x \leftarrow z\} \rangle\{y \leftarrow w\} \triangleleft R$$

The idea is to delay the merging of t and $O'\langle z \rangle$ as $O'\langle t\{x \leftarrow z\} \rangle$
 $\hookrightarrow$ put the pair (**slice**) (t, x) , denoted as $t[x \leftarrow \cdot]$, in a new stack storing delayed merges, and keeping $O'\langle z \rangle\{y \leftarrow w\}$ as the active code.

This leads to the following **Sliced PPositive Machine (Sliced POM)**.

The $\rightsquigarrow_m$ transition of the natural POM is replaced by:

$$\begin{array}{c} S \\ S : t[x \leftarrow \cdot] \end{array} \left| \begin{array}{c} t[x \leftarrow (\lambda y. u)w] \\ z \end{array} \right| \begin{array}{c} E \\ E \end{array} \parallel \begin{array}{c} \rightsquigarrow_m \\ \rightsquigarrow_{\text{sea}_3} \end{array} \parallel \begin{array}{c} S : t[x \leftarrow \cdot] \\ S \end{array} \left| \begin{array}{c} u\{y \leftarrow w\} \\ t\{x \leftarrow z\} \end{array} \right| \begin{array}{c} E \\ E \end{array}$$

Sliced POM

S	$t[x \leftarrow \lambda y.u]$	E	$\leadsto_{\text{sea}_1}$	S	t	$[x \leftarrow \lambda y.u] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_{\text{sea}_2}$	S	t	$[x \leftarrow yz] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_e$	S	$t[x \leftarrow (\lambda w.u)^\alpha z]$	E
S	$t[x \leftarrow (\lambda y.u)w]$	E	$\leadsto_m$	S : $t[x \leftarrow \cdot]$	$u\{y \leftarrow w\}$	E
S : $t[x \leftarrow \cdot]$	z	E	$\leadsto_{\text{sea}_3}$	S	$t\{x \leftarrow z\}$	E

Sliced POM

S	$t[x \leftarrow \lambda y.u]$	E	$\leadsto_{\text{sea}_1}$	S	t	$[x \leftarrow \lambda y.u] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_{\text{sea}_2}$	S	t	$[x \leftarrow yz] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_e$	S	$t[x \leftarrow (\lambda w.u)^\alpha z]$	E
S	$t[x \leftarrow (\lambda y.u)w]$	E	$\leadsto_m$	S : $t[x \leftarrow \cdot]$	$u\{y \leftarrow w\}$	E
S : $t[x \leftarrow \cdot]$	z	E	$\leadsto_{\text{sea}_3}$	S	$t\{x \leftarrow z\}$	E

Lemma (Sub-term property)

Let $t \triangleleft Q \leadsto^* Q' = (S, u, E)$ be a Sliced POM run. Then:

1. If Q' is not the target of an e-transition then $|r| \leq |t|$ for any term r in Q' ;
2. Otherwise, $|r| \leq |t|$ for any term r in Q' except u .

Sliced POM

S	$t[x \leftarrow \lambda y.u]$	E	$\leadsto_{\text{sea}_1}$	S	t	$[x \leftarrow \lambda y.u] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_{\text{sea}_2}$	S	t	$[x \leftarrow yz] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_e$	S	$t[x \leftarrow (\lambda w.u)^\alpha z]$	E
S	$t[x \leftarrow (\lambda y.u)w]$	E	$\leadsto_m$	S : $t[x \leftarrow \cdot]$	$u\{y \leftarrow w\}$	E
S : $t[x \leftarrow \cdot]$	z	E	$\leadsto_{\text{sea}_3}$	S	$t\{x \leftarrow z\}$	E

Lemma (Sub-term property)

Let $t \triangleleft Q \leadsto^* Q' = (S, u, E)$ be a Sliced POM run. Then:

1. If Q' is not the target of an e-transition then $|r| \leq |t|$ for any term r in Q' ;
2. Otherwise, $|r| \leq |t|$ for any term r in Q' except u .

Lemma (Cost of single transitions)

Let $t \triangleleft Q \leadsto^* Q'$ be a Sliced POM run. Implementing sea_1 and sea_2 transitions from Q' costs $\mathcal{O}(1)$ each while implementing e , m , and sea_3 transitions costs $\mathcal{O}(|t|)$ each.

Sliced POM

S	$t[x \leftarrow \lambda y.u]$	E	$\rightsquigarrow_{\text{sea}_1}$	S	t	$[x \leftarrow \lambda y.u] : E$
S	$t[x \leftarrow yz]$	E	$\rightsquigarrow_{\text{sea}_2}$	S	t	$[x \leftarrow yz] : E$
S	$t[x \leftarrow yz]$	E	$\rightsquigarrow_e$	S	$t[x \leftarrow (\lambda w.u)^\alpha z]$	E
S	$t[x \leftarrow (\lambda y.u)w]$	E	$\rightsquigarrow_m$	S : $t[x \leftarrow \cdot]$	$u\{y \leftarrow w\}$	E
S : $t[x \leftarrow \cdot]$	z	E	$\rightsquigarrow_{\text{sea}_3}$	S	$t\{x \leftarrow z\}$	E

Lemma (Number of transitions)

Let $r : t \triangleleft Q \rightsquigarrow^* Q'$ be a Sliced POM run.

1. $|r|_{e, \text{sea}_3} \in \mathcal{O}(|r|_m)$;
2. $|r|_{\text{sea}_1, \text{sea}_2} \in \mathcal{O}(|t| \cdot (|r|_m + 1))$.

Lemma (Cost of single transitions)

Let $t \triangleleft Q \rightsquigarrow^* Q'$ be a Sliced POM run. Implementing sea_1 and sea_2 transitions from Q' costs $\mathcal{O}(1)$ each while implementing e , m , and sea_3 transitions costs $\mathcal{O}(|t|)$ each.

Sliced POM

S	$t[x \leftarrow \lambda y.u]$	E	$\leadsto_{\text{sea}_1}$	S	t	$[x \leftarrow \lambda y.u] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_{\text{sea}_2}$	S	t	$[x \leftarrow yz] : E$
S	$t[x \leftarrow yz]$	E	$\leadsto_e$	S	$t[x \leftarrow (\lambda w.u)^\alpha z]$	E
S	$t[x \leftarrow (\lambda y.u)w]$	E	$\leadsto_m$	$S : t[x \leftarrow \cdot]$	$u\{y \leftarrow w\}$	E
$S : t[x \leftarrow \cdot]$	z	E	$\leadsto_{\text{sea}_3}$	S	$t\{x \leftarrow z\}$	E

Theorem

Let $r : t \triangleleft Q \leadsto^* Q'$ be a Sliced POM run. Then r can be implemented on random access machines in $\mathcal{O}(|t| \cdot (|r|_m + 1))$.

Conclusions and future work

We have provided an efficient implementation of λ_{pos} using slice stacks that are dual to the usual structure of machine environments, justifying its value as a tool for the study of sharing.

Conclusions and future work

We have provided an efficient implementation of λ_{pos} using slice stacks that are dual to the usual structure of machine environments, justifying its value as a tool for the study of sharing.

Some future directions:

- Adapt λ_{pos} and the sliced POM to call-by-need
- Combine the slicing technique with other implementation techniques on abstract machines

References



Accattoli, B. and Wu, J.-H. (2024).

Positive focusing is directly useful.

In 40th Conference on Mathematical Foundations of Programming Semantics (MFPS 2024).



Miller, D. and Wu, J.-H. (2023).

A positive perspective on term representation.

In 31st EACSL Annual Conference on Computer Science Logic (CSL 2023).



Wu, J.-H. (2023).

Proofs as terms, terms as graphs.

In Asian Symposium on Programming Languages and Systems (APLAS 2023), pages 91–111. Springer.

Thank you for your listening!